3-Day DSA Interview Crash Plan

Goal

You already have a base in DSA. This plan is designed to:

- Revise core concepts quickly
- Build interview pattern recognition
- Improve coding speed
- Focus on recursion, trees, BST, trie, heap, heap sort, and graphs
- Prepare you for coding + explanation rounds

Overall Strategy

What You Should Do Every Day

1. Learn Pattern → Not Just Problems

For every topic ask:

- What is the traversal pattern?
- What is the recursive relation?
- What changes in edge cases?
- What is the time complexity?
- Can I optimize space?

---

2. Solve in This Order

1. Brute force
2. Better approach
3. Optimal approach
4. Explain complexity
5. Dry run manually

---

3. Interview Template

Whenever solving:

```
// 1. Clarify input
// 2. Discuss brute force
// 3. Explain optimized idea
// 4. Write code
```

```
// 5. Dry run
// 6. Mention TC & SC
```

DAY 1 — Recursion + Tree + Binary Tree

Total Time: ~5–6 Hours

PART 1 — Recursion Fundamentals (1.5 Hours)

Focus Areas

Understand:

- Base condition
- Recursive hypothesis
- Faith & induction idea
- Stack memory
- Backtracking flow

Master These Patterns

1. Print Pattern Recursion

- print 1 to n
- print n to 1
- factorial
- fibonacci

2. Parameterized vs Functional Recursion

```
// Parameterized
function sum(n, total)

// Functional
function sum(n)
```

3. Backtracking

- reverse array
- palindrome check
- generate subsequences

MUST DO QUESTIONS

Easy

1. Factorial
2. Fibonacci
3. Reverse array
4. Check palindrome
5. Sum of N numbers

Medium

1. Print all subsequences
2. Subset sum
3. Combination sum
4. Permutations
5. N-Queens (understand only)

Key Interview Insight

Whenever recursion question comes:

Ask yourself:

```
1. What is reducing?
2. What is base case?
3. What is recursive call doing?
4. What should current function return?
```

PART 2 — Binary Tree Basics (2 Hours)

Important Theory

Tree Terminologies

- Root
- Leaf
- Height
- Depth
- Level
- Parent
- Child
- Sibling

Binary Tree Traversals (VERY IMPORTANT)

DFS Traversals

Preorder

```
Root Left Right
```

Inorder

```
Left Root Right
```

Postorder

```
Left Right Root
```

Learn Both

- Recursive traversals
- Iterative traversals using stack

BFS Traversal

Level Order Traversal

Use queue.

Pattern:

```
while(queue.length)
```

MUST DO QUESTIONS

Easy

1. Preorder traversal

2. Inorder traversal
3. Postorder traversal
4. Level order traversal
5. Count nodes
6. Height of tree
7. Max depth
8. Search in tree

Medium

1. Diameter of tree
2. Balanced binary tree
3. Left view
4. Right view
5. Zigzag traversal
6. Boundary traversal
7. Lowest common ancestor
8. Check identical trees

VERY IMPORTANT INTERVIEW PATTERNS

Pattern 1 — DFS Recursive Template

```
function dfs(root) {
   if(!root) return;

   dfs(root.left);
   dfs(root.right);
}
```

Pattern 2 — Level Order BFS

```
let queue = [root];

while(queue.length) {
   let size = queue.length;

   for(let i=0; i<size; i++) {
      let node = queue.shift();
   }
}
```

PART 3 — Binary Search Tree (1.5 Hours)

MOST IMPORTANT PROPERTY

```
Left < Root < Right
```

This single property solves most BST questions.

MUST KNOW OPERATIONS

1. Search

```
if(val < root.val)
```

2. Insert

3. Delete

4. Validate BST

5. Find Min/Max

MUST DO QUESTIONS

Easy

1. Search in BST
2. Insert into BST
3. Minimum in BST
4. Maximum in BST
5. Validate BST

Medium

1. Delete node in BST
2. Kth smallest
3. LCA in BST
4. BST iterator
5. Convert sorted array to BST

MOST ASKED INTERVIEW CONCEPT

Inorder of BST is sorted

This helps in:

- Kth smallest
- Validation
- Sorted output
- Range queries

End of Day 1 Revision

Revise Before Sleep

You should be able to explain:

- Difference between recursion & backtracking
- DFS vs BFS
- Tree vs BST
- Why inorder is sorted in BST
- Recursive traversal code from memory

DAY 2 — Trie + Heap + Heap Sort

Total Time: ~5 Hours

PART 1 — Trie (1.5 Hours)

Core Idea

Trie stores characters level by level.

Used for:

- Autocomplete
- Prefix search
- Dictionary
- Word search

Understand Trie Node

```
class TrieNode {
   constructor() {
      this.children = {};
      this.end = false;
   }
}
```

MUST KNOW OPERATIONS

1. Insert Word

2. Search Word

3. StartsWith Prefix

MUST DO QUESTIONS

Easy

1. Implement Trie
2. Search word
3. Prefix matching

Medium

1. Word dictionary
2. Replace words
3. Longest common prefix
4. Word search II (understand)

Interview Trick

Trie questions usually involve:

```
prefix
string matching
```

```
dictionary
autocomplete
```

PART 2 — Heap (2 Hours)

Core Concept

Heap is a complete binary tree.

Types

Max Heap

Parent bigger than children.

Min Heap

Parent smaller than children.

MUST KNOW

Heap Operations

Insert

Delete

Heapify

Extract Min/Max

Important Array Relations

```
left = 2*i + 1
right = 2*i + 2
parent = Math.floor((i-1)/2)
```

MOST IMPORTANT INTERVIEW USE CASES

Use Heap When:

1. K largest/smallest

2. Top K frequent

3. Priority processing

4. Streaming data

5. Median problems

MUST DO QUESTIONS

Easy

1. Implement min heap
2. Heapify array
3. Kth largest element

Medium

1. Top K frequent elements
2. K closest points
3. Merge K sorted arrays
4. Task scheduler
5. Median from data stream

PART 3 — Heap Sort (1 Hour)

Core Idea

1. Build max heap
2. Swap root with last
3. Heapify again
4. Repeat

Complexity

```
Time: O(n log n)
Space: O(1)
```

MUST UNDERSTAND

Why heap sort is in-place

Why quick sort is usually faster practically

Heap construction complexity = O(n)

End of Day 2 Revision

You should be able to explain:

- Heap vs BST
- Heap vs Priority Queue
- Why heap is complete binary tree
- When to use Trie
- Heap insertion/deletion process

DAY 3 — Graphs + Interview Revision

Total Time: ~6 Hours

PART 1 — Graph Fundamentals (1.5 Hours)

MUST KNOW REPRESENTATIONS

Adjacency List

```
let graph = {
   0:[1,2],
```

```
    1:[0,3]
}
```

Adjacency Matrix

Understand

- Directed graph
- Undirected graph
- Weighted graph
- Cyclic graph
- DAG
- Connected components

PART 2 — Graph Traversals (2 Hours)

DFS

Recursive DFS

```
function dfs(node) {
   visited[node] = true;

   for(let nei of graph[node]) {
      if(!visited[nei]) {
          dfs(nei);
      }
   }
}
```

BFS

Queue Based

Used for shortest path in unweighted graph.

MUST DO QUESTIONS

Easy

1. DFS traversal
2. BFS traversal
3. Number of islands
4. Flood fill
5. Connected components

Medium

1. Detect cycle in undirected graph
2. Detect cycle in directed graph
3. Rotten oranges
4. Clone graph
5. Bipartite graph

PART 3 — Advanced Graph Patterns (1.5 Hours)

MUST KNOW ALGORITHMS

1. Topological Sort

Used in:

- dependency problems
- course schedule

---

2. Dijkstra

Used for shortest path with weights.

Uses min heap.

---

3. Union Find / DSU

Used in:

- cycle detection
- connected components

MUST DO QUESTIONS

1. Course schedule
2. Network delay time
3. Cheapest flights
4. Kruskal algorithm
5. Number of provinces

FINAL INTERVIEW REVISION (1 Hour)

MOST IMPORTANT INTERVIEW QUESTIONS

Trees

- DFS vs BFS
- Balanced tree
- Diameter
- LCA
- BST validation

Heap

- Kth largest
- Top K frequent
- Priority queue use cases

Trie

- Prefix search
- Autocomplete

Graph

- BFS vs DFS
- Cycle detection
- Topological sort
- Dijkstra

HIGH-YIELD INTERVIEW TIPS

1. Always Speak While Coding

Explain:

- why using recursion
- why queue/stack
- why heap
- complexity

---

2. Never Jump Directly to Code

First discuss:

```
Brute force
Better approach
Optimal approach
```

---

3. Common Mistakes

Recursion

- forgetting base case
- stack overflow

Tree

- null checks
- left/right confusion

Graph

- forgetting visited array
- infinite traversal

Heap

- wrong heapify direction

Best Practice Platform Order

Priority Questions

LeetCode

- Easy → Medium

NeetCode Roadmap

Excellent for patterns.

Striver A2Z DSA Sheet

Best for interview revision.

Ultra Important Patterns Summary

Topic	Core Pattern
Recursion	base case + smaller problem
Tree DFS	recursive traversal
Tree BFS	queue level traversal
BST	left < root < right
Trie	prefix character traversal
Heap	parent-child ordering
Graph DFS	recursive exploration
Graph BFS	shortest path in unweighted graph

Final Night Before Interview

Revise ONLY:

1. Traversal templates
2. Complexity analysis
3. Common patterns
4. Dry runs
5. Edge cases

Do NOT start new hard problems.

Golden Rule

In interviews:

```
Communication > Perfect Code
```

Even if stuck:

- explain thought process
- discuss edge cases
- propose optimizations
- keep speaking

That alone creates strong interviewer impression.